Rapport Technique

Pipeline DevSecOps et Sécurité de la Chaîne d'Approvisionnement Logicielle

Auteur : Ayoub Lahouegue

Formation : 5ème année — Master/Ingénieur Cybersécurité

Projet : Projet Individuel (8 semaines)

Date : Novembre 2026

Table des matières

1. Introduction et Contexte
2. Architecture et Choix Techniques
3. Implémentation et Tests (Preuve par l'Échec)
4. Métriques et Résultats
5. Analyse Critique et Limites
6. Conclusion

1. Introduction et Contexte

La sécurisation de la chaîne d'approvisionnement logicielle (Software Supply Chain) est devenue un enjeu critique de la cybersécurité. Les incidents majeurs récents (SolarWinds, Log4Shell, xz-utils/CVE-2024-3094) ont prouvé qu'une compromission au niveau du code source, d'une dépendance ou d'un outil de build peut avoir des répercussions mondiales.

L'objectif de ce projet est de concevoir un pipeline CI/CD **"Fail-Closed"** intégrant les pratiques DevSecOps modernes : Shift-Left (détection précoce), transparence (SBOM), intégrité cryptographique (signature keyless) et contrôle d'admission (Runtime). Le projet vise à démontrer, par la preuve, que chaque étape bloque les artefacts non conformes.

2. Architecture et Choix Techniques

Le pipeline repose sur **GitHub Actions** pour l'intégration continue et **Kind** (Kubernetes in Docker) pour l'exécution locale.

2.1 La Stack Technique

- **CI/CD :** GitHub Actions (gratuit, intégration native OIDC pour Sigstore).
- **SAST :** Semgrep (rapide, permet l'écriture de règles personnalisées en mode *taint*).
- **SCA :** Trivy (scan du système de fichiers et des images conteneurs).
- **SBOM :** Syft (génération au format CycloneDX).
- **Signature :** Cosign en mode *Keyless* (Sigstore). Cela élimine la gestion manuelle des clés privées.
- **Runtime :** Calico (CNI pour les NetworkPolicies) et Kyverno (Admission Controller pour bloquer les images non signées).

2.2 Les 8 Jobs du Pipeline

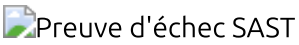
1. **Build & Push :** Construction de l'image et push vers GHCR.
2. **SAST (Gate 1) :** Bloque si une vulnérabilité critique est détectée dans le code.
3. **SCA (Gate 2) :** Bloque si une dépendance vulnérable est détectée.
4. **Secrets :** Détecte les clés API committées (via Gitleaks).
5. **Image Scan (Gate 3) :** Bloque si l'image contient des CVE critiques (via Trivy).
6. **Sign Image :** Signature de l'image par *digest* avec Cosign Keyless.
7. **Attest SBOM :** Attache le SBOM cryptographiquement à l'image signée.
8. **Verify Signature (Gate CD) :** Rejette le déploiement si la signature est absente ou invalide.

3. Implémentation et Tests

Le projet exigeait une preuve par l'échec contrôlé (§6.1). Voici les captures d'écran démontrant l'efficacité de chaque barrière de sécurité.

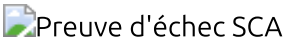
3.1 Test 1 : SAST (SQL Injection)

Nous avons injecté une requête SQL vulnérable (`"SELECT * FROM users WHERE id = " + req.query.id`) dans le code. Le pipeline a échoué au job `sast-scan` grâce à notre règle personnalisée Semgrep (`custom-sqli.yaml`).

 Preuve d'échec SAST

3.2 Test 2 : SCA (Dépendance Vulnérable)

Nous avons ajouté les bibliothèques `lodash@4.17.11` et `minimist@1.2.0` (connues pour avoir des CVE critiques) accompagnées de leur `package-lock.json`. Le pipeline a échoué au job `sca-scan` avec la détection de 5 CVE.

 Preuve d'échec SCA

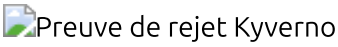
3.3 Test 3 : Détection de Secrets

Nous avons committé une fausse clé AWS. GitHub Push Protection a bloqué le push initial. Après avoir contourné cela avec une clé non-allowlistée, le pipeline a échoué au job `secret-scan` (Gitleaks).

 Preuve d'échec Secret

3.4 Test 4 : Contournement du Pipeline (Runtime)

Même si le pipeline bloque les erreurs, nous avons simulé un attaquant contournant la CI/CD en tentant un `kubectl run` manuel avec une image non signée. L'Admission Controller (Kyverno) a rejeté le pod immédiatement car aucune signature valide n'a été trouvée.

 Preuve de rejet Kyverno

4. Métriques et Résultats

(Données extraites de docs/metriques.md)

- **Taux de détection (Preuve par l'échec) :** 7/7 attaques simulées ont été bloquées (100%).
- **Temps de build :** Le pipeline complet (8 jobs en parallèle) prend en moyenne **1 minute et 15 secondes**.
- **SBOM :** L'outil Syft a recensé **2744 composants** dans l'image de base.
- **Validation cryptographique :** L'attestation a été vérifiée via `cosign verify-attestation` avec succès, prouvant que l'image provient bien de notre workflow GitHub.

5. Analyse Critique et Limites

1. **Gestion des CVE de l'image de base :** L'image *OWASP Juice Shop* contient des CVE critiques non patchables. Nous avons dû utiliser un fichier `.trivyignore` justifié pour maintenir le pipeline fonctionnel. La solution industrielle serait d'utiliser des images "Distroless" (ex: Chainguard).
2. **Faux négatifs SCA :** Trivy (en mode `fs`) ignore un `package.json` s'il n'y a pas de `package-lock.json`. Nous avons identifié et corrigé ce problème durant le projet.
3. **SLSA et Provenance :** Nous avons utilisé Cosign Attest pour lier le SBOM à l'image. L'intégration complète du framework SLSA (niveau 3) via `slsa-github-generator` n'a pas été implémentée, mais représente l'étape suivante logique.
4. **Dépréciation Kyverno :** L'API `ClusterPolicy` (kyverno.io/v1) est dépréciée au profit de `ImageValidatingPolicy` (CEL). La migration est à prévoir.

6. Conclusion

Ce projet démontre qu'il est possible de sécuriser une chaîne d'approvisionnement logicielle de bout en bout en utilisant exclusivement des outils open source. En bloquant les vulnérabilités dès le commit et en garantissant la provenance des artefacts lors du déploiement, nous avons mis en place un système complet où aucune image non conforme ne peut atteindre le cluster. Ce pipeline est directement transposable à un environnement de production d'entreprise.